# LeetCode – Nov 2026 | Day 3

**Author:** Eric Zhou
**Email:** eric_ds_zhou@outlook.com
**Date:** Nov 8, 2026

## Introduction

Today, we will review LeetCode 27 and work on LeetCode 80. Both problems use the Two Pointers technique to modify an array in-place. LeetCode 27 reviews the basic approach, while LeetCode 80 extends the same idea by introducing additional conditions for handling duplicates.

## Leetcode 27 - Remove Element

### Question

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` from `nums` in-place.

The order of the remaining elements may be changed. Then return the number of elements that are not equal to `val`.

Consider the number of elements in nums which are not equal to `val` be `k`, to get accepted, you need to do the following things:

Change the array `nums` such that the first `k` elements of nums contain the elements which are not equal to `val`.

The remaining elements of `nums` are not important as well as the size of nums.

Return `k`.

### Intuition

We can use the **Two Pointers** technique without creating another array. One pointer traverses the array while the other tracks where the next valid element should go.

When an element does not equal `val`, we keep it by moving it to the beginning of the array. Otherwise, we skip it.

### Approach

We use two pointers:

- `i`: The Fast Pointer that traverses the entire array.
- `result`: The Slow Pointer that indicates the next position for a valid element.

Initially, `result = 0`.

1. Check whether `nums[i] != val`.
2. If true, assign `nums[i]` to `nums[result]` and increment `result`.

3. Otherwise, skip the current element.
4. After the loop, return `result`.

Since `result` never exceeds `i`, writing to `nums[result]` does not overwrite an element that has not yet been checked.

### Time Complexity

$$\boxed{\Theta(n)}$$

We traverse the entire array once. Every iteration takes constant time, so the total time complexity is $\Theta(n)$.

### Space Complexity

$$\boxed{\Theta(1)}$$

We use only a constant amount of extra space, regardless of the array size.

### Code

```java
class Solution {
    public int removeElement(int[] nums, int val) {
        int result = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[result] = nums[i];
                result++;
            }
        }

        return result;
    }
}
```

---

*Since LeetCode 27 is very similar to LeetCode 26, we only briefly reviewed it today. Our main focus is LeetCode 80, which builds on the same Two Pointers technique with an additional condition for handling duplicates.*

## Leetcode 80 - Remove Duplicates from Sorted Array II

### Question

Given an integer array `nums` sorted in non-decreasing order, remove duplicates in-place so that each unique element appears **at most twice**.

The relative order of the remaining elements must be preserved. Return the number of remaining elements, `k`.

The first `k` elements of `nums` should contain the valid elements.

Return k after placing the final result in the first k slots of nums.

Do not allocate extra space for another array. You must do this by modifying the input array in-place with O(1) extra memory.

## Intuition

This problem extends the Two Pointers technique from LeetCode 27. Instead of removing a particular value, we need to keep no more than two occurrences of each number.

Because the array is sorted, duplicates occur consecutively. We can compare the current number with the **second-to-last retained element**, `nums[result - 2]`. If the current number is greater, it is safe to retain it.

## Approach

We use two pointers:

- `i`: The Fast Pointer that traverses the array.
- `result`: The Slow Pointer that indicates where the next valid element should be written.

If the array contains at most two elements, return its length immediately.

Otherwise, initialize `result = 2`, since the first two elements can always be retained. Starting from `i = 2`, check:

```
if (nums[i] > nums[result - 2])
```

**Case 1:** `nums[i] == nums[result - 2]`

Because the array is sorted, the current number equals the second-to-last retained number. Keeping it would create at least three occurrences, so we skip it.

**Case 2:** `nums[i] > nums[result - 2]`

The current number differs from the second-to-last retained number. We can safely write it to `nums[result]` and increment `result`.

Finally, return `result`.

## Time Complexity

$$\boxed{\Theta(n)}$$

We traverse the array once, and each iteration performs constant work. Therefore, the total time complexity is $\Theta(n)$.

## Space Complexity

$$\boxed{\Theta(1)}$$

We modify the input array in-place using only a constant amount of extra space.

## Code

```java
class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length <= 2) {
            return nums.length;
        }

        int result = 2;

        for (int i = 2; i < nums.length; i++) {
            if (nums[i] > nums[result - 2]) {
                nums[result] = nums[i];
                result++;
            }
        }

        return result;
    }
}
```